

EXP-3(GAN)

```
[2] ✓ 10s
import tensorflow as tf
from tensorflow.keras import layers
import numpy as np
import matplotlib.pyplot as plt

[3] ✓ 2s
(train_images, _), (_, _) = tf.keras.datasets.mnist.load_data()
train_images = (train_images.reshape(-1, 28, 28, 1).astype('float32') - 127.5) / 127.5
train_dataset = tf.data.Dataset.from_tensor_slices(train_images).shuffle(60000).batch(256)

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 1s 0us/step

[4] ✓ 0s
def make_generator():
    return tf.keras.Sequential([
        layers.Dense(7*7*256, use_bias=False, input_shape=(100,)),
        layers.BatchNormalization(), layers.LeakyReLU(),
        layers.Reshape((7, 7, 256)),
        layers.Conv2DTranspose(128, 5, strides=1, padding='same', use_bias=False),
        layers.BatchNormalization(), layers.LeakyReLU(),
        layers.Conv2DTranspose(64, 5, strides=2, padding='same', use_bias=False),
        layers.BatchNormalization(), layers.LeakyReLU(),
        layers.Conv2DTranspose(1, 5, strides=2, padding='same', use_bias=False, activation='tanh')
    ])

[5] ✓ 1s
def make_discriminator():
    return tf.keras.Sequential([
        layers.Conv2D(64, 5, strides=2, padding='same', input_shape=[28, 28, 1]),
        layers.LeakyReLU(), layers.Dropout(0.3),
        layers.Conv2D(128, 5, strides=2, padding='same'),
        layers.LeakyReLU(), layers.Dropout(0.3),
        layers.Flatten(), layers.Dense(1)
    ])

generator = make_generator()
discriminator = make_discriminator()

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarnin
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

[6] ✓ 0s
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)
gen_opt = tf.keras.optimizers.Adam(1e-4)
disc_opt = tf.keras.optimizers.Adam(1e-4)

def gen_loss(fake_out): return cross_entropy(tf.ones_like(fake_out), fake_out)
def disc_loss(real_out, fake_out):
    return cross_entropy(tf.ones_like(real_out), real_out) + cross_entropy(tf.zeros_like(fake_out),
```

EXP-3(GAN)

```
17] 11m @tf.function
def train_step(images):
    noise = tf.random.normal([256,100])
    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        fake_images = generator(noise, training=True)
        real_out = discriminator(images, training=True)
        fake_out = discriminator(fake_images, training=True)
        gen_loss_val, disc_loss_val = gen_loss(fake_out), disc_loss(real_out, fake_out)
        gen_grad = gen_tape.gradient(gen_loss_val, generator.trainable_variables)
        disc_grad = disc_tape.gradient(disc_loss_val, discriminator.trainable_variables)
        gen_opt.apply_gradients(zip(gen_grad, generator.trainable_variables))
        disc_opt.apply_gradients(zip(disc_grad, discriminator.trainable_variables))

    # Training loop
    def train(dataset, epochs):
        for e in range(epochs):
            for batch in dataset: train_step(batch)
            if (e+1)%10==0: generate_images(generator, e+1)
            print(f'Epoch {e+1} done')

def generate_images(model, epoch):
    noise = tf.random.normal([16,100])
    imgs = model(noise, training=False)
    plt.figure(figsize=(4,4))
    for i in range(16):
        plt.subplot(4,4,i+1)
        plt.imshow(imgs[i,:,:,0]*127.5+127.5, cmap='gray')
        plt.axis('off')
    plt.suptitle(f'Epoch {epoch}')
    plt.show()

# Train GAN
train(train_dataset, 50)

# Show Real vs Fake
plt.figure(figsize=(4,4)); plt.suptitle('Real MNIST')
for i in range(16): plt.subplot(4,4,i+1); plt.imshow(train_images[i,:,:,0]*127.5+127.5, cmap='gray'); plt.axis('off')
plt.show()

noise = tf.random.normal([16,100]); fake_imgs = generator(noise, training=False)
plt.figure(figsize=(4,4)); plt.suptitle('Generated Images')
for i in range(16): plt.subplot(4,4,i+1); plt.imshow(fake_imgs[i,:,:,0]*127.5+127.5, cmap='gray'); plt.axis('off')
plt.show()

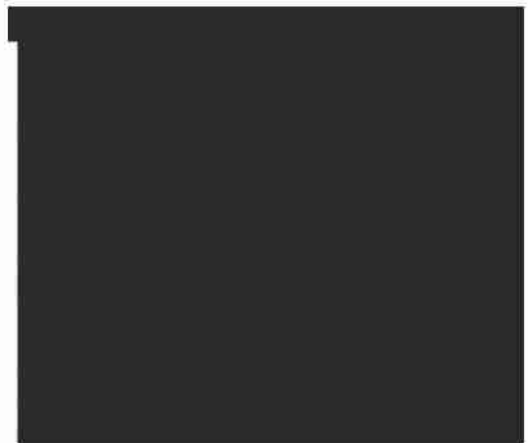
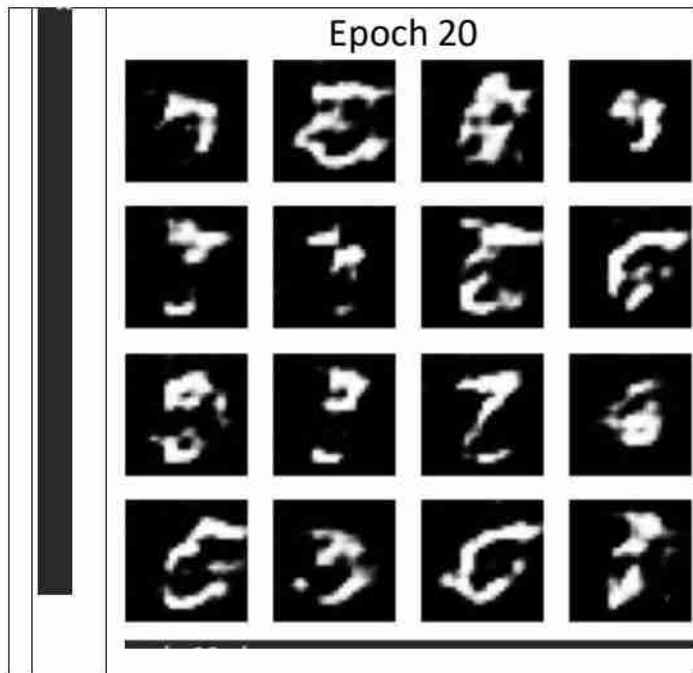
Epoch 1 done
Epoch 2 done
Epoch 3 done
Epoch 4 done
Epoch 5 done
Epoch 6 done
Epoch 7 done
Epoch 8 done
Epoch 9 done
Epoch 10
```



EXP-3(GAN)



EXP-3(GAN)



EXP-3(GAN)

Epoch 20

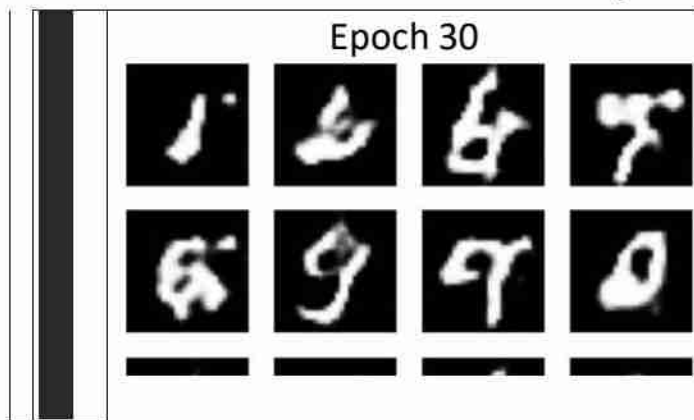


Epoch 20 done
Epoch 21 done
Epoch 22 done
Epoch 23 done
Epoch 24 done
Epoch 25 done
Epoch 26 done
Epoch 27 done
Epoch 28 done
Epoch 29 done

Epoch 30

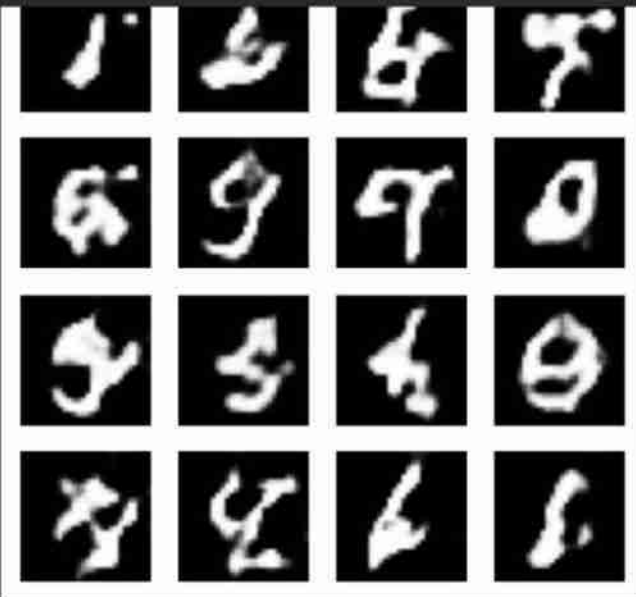


EXP-3(GAN)



EXP-3(GAN)

...

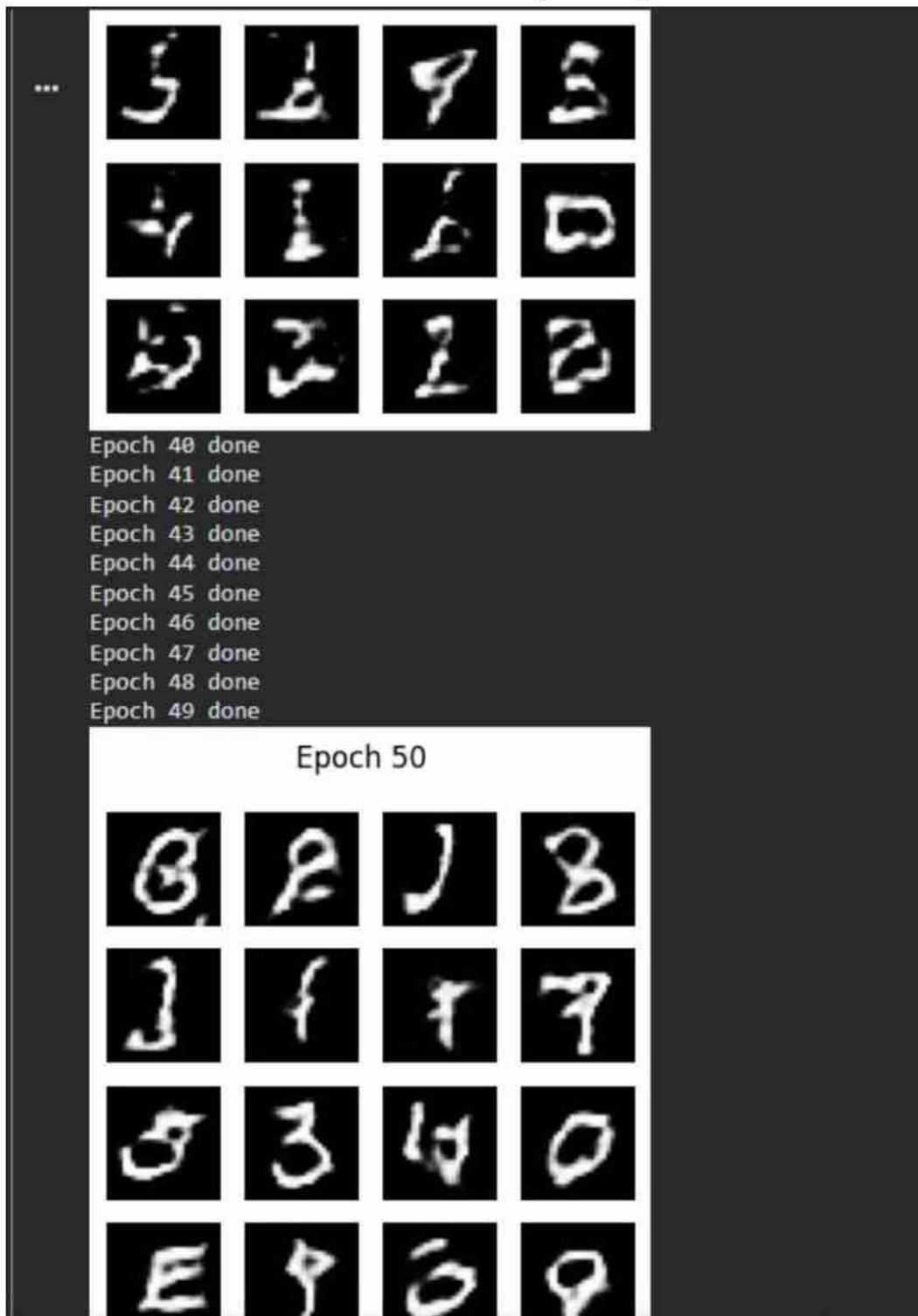


Epoch 30 done
Epoch 31 done
Epoch 32 done
Epoch 33 done
Epoch 34 done
Epoch 35 done
Epoch 36 done
Epoch 37 done
Epoch 38 done
Epoch 39 done

Epoch 40

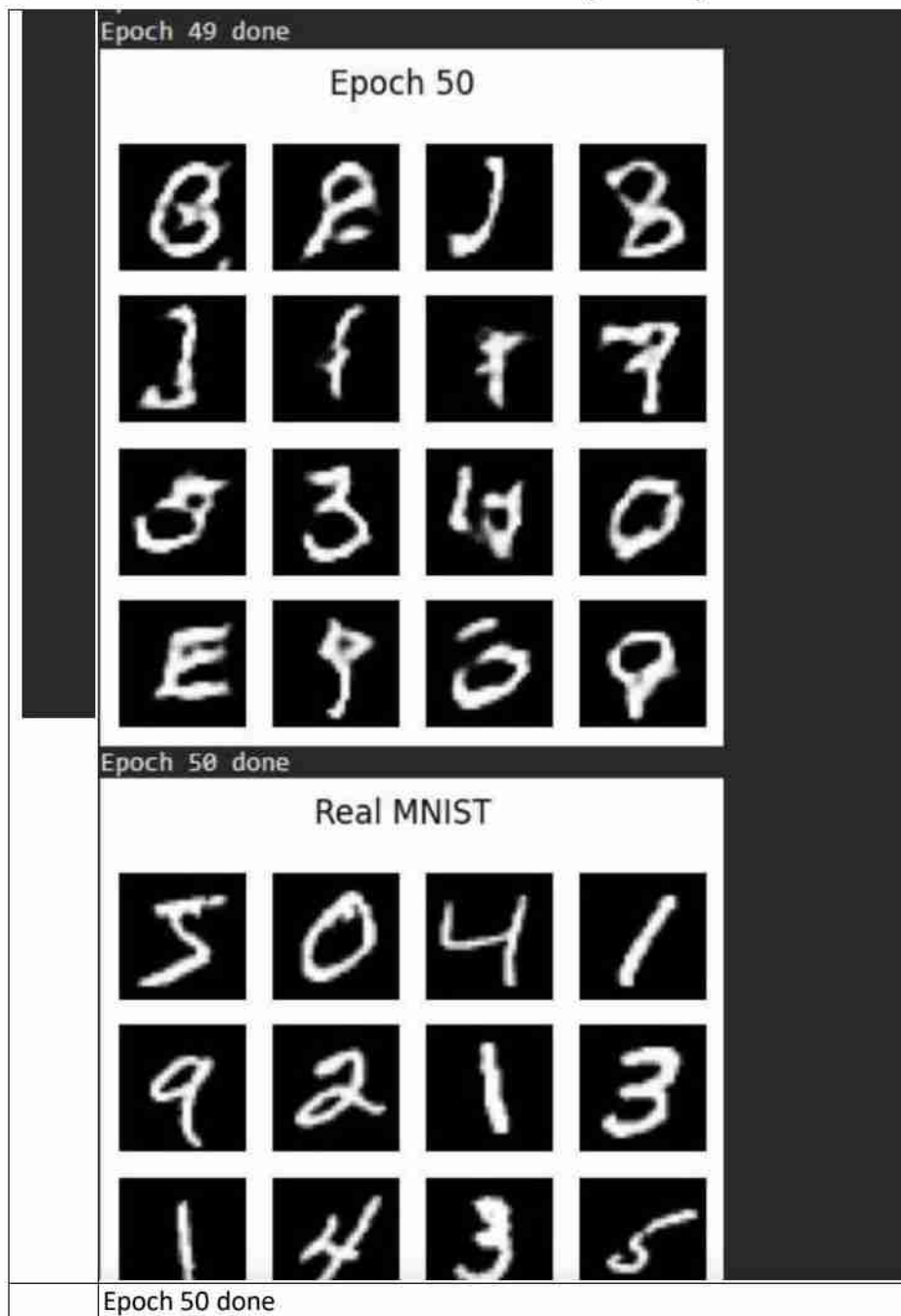


EXP-3(GAN)

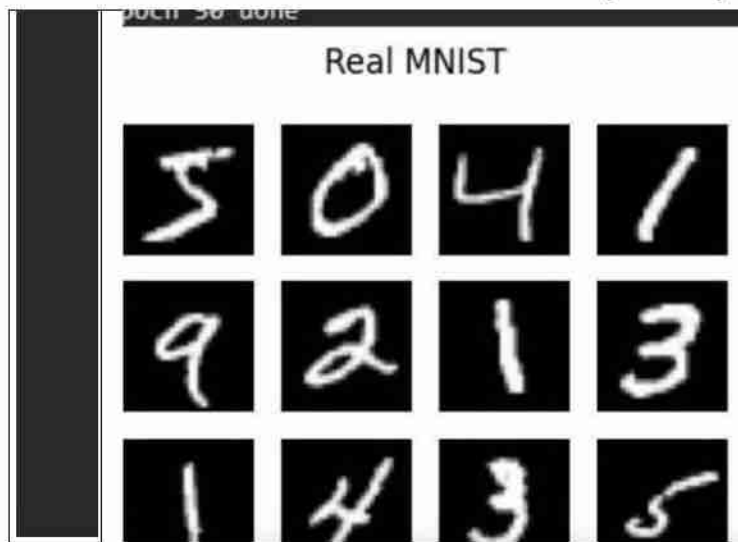


Epoch 49 done

EXP-3(GAN)



EXP-3(GAN)



nu

36

